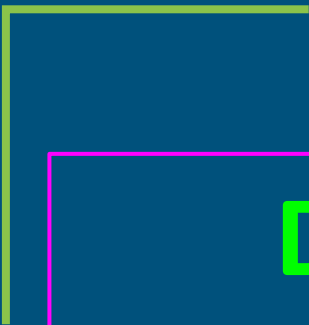
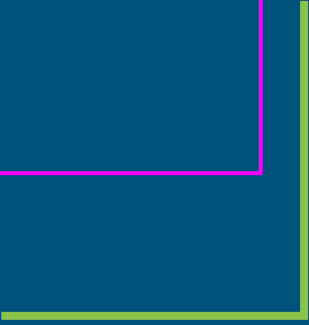




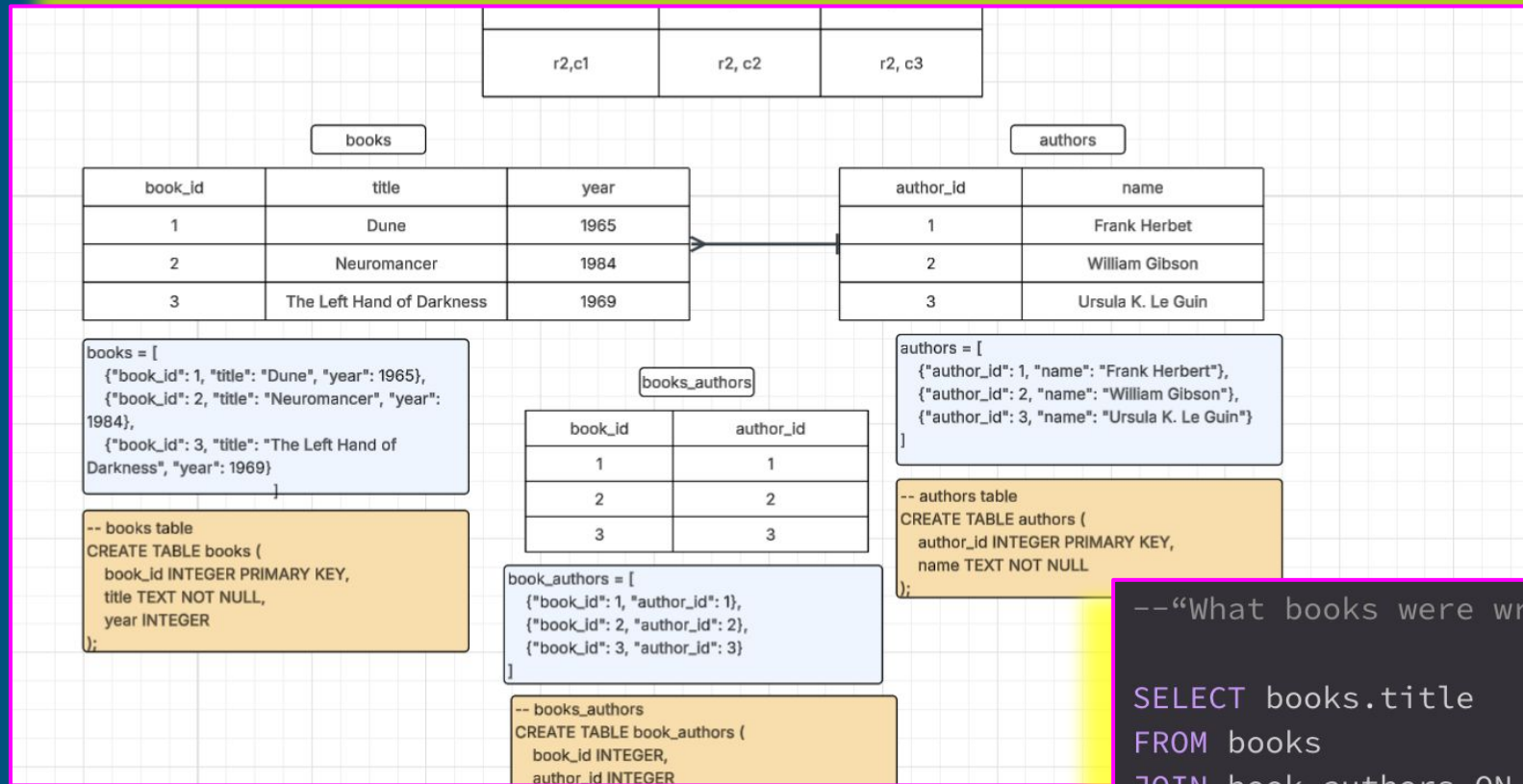
Database Design & Web Implementation CSCI-UA060



Relationship Modeling



It's common for multiple tables to be relevant in a database

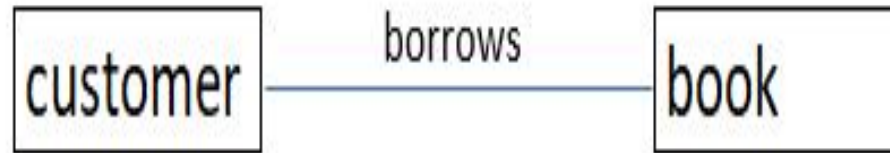


--"What books were written by Ursula K. Le Guin?"

```
SELECT books.title
FROM books
JOIN book_authors ON books.book_id = book_authors.book_id
JOIN authors ON authors.author_id = book_authors.author_id
WHERE authors.name = 'Ursula K. Le Guin';
```

Relationships

- Relationships describe how one table connects to another



Database Design Goals

- **Integrity:** Data integrity, Correctness, Constraints, No contradictory data
- **Clarity :** Schema readability / maintainability, Clear table names, Clear column meanings, Predictable structure
- **Resolve ambiguity:** Normalization + explicit relationships, Avoid duplicated meanings, Avoid “sometimes this, sometimes that” columns, Make relationships explicit instead of implied

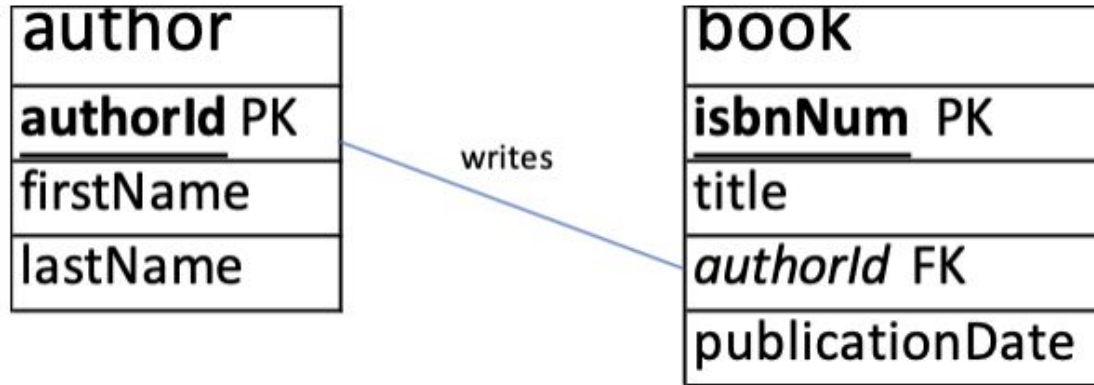
Primary Key

- In each table, there must be one column (or group of columns) that can uniquely identify each row. For a person, that might be a social security #. For a book, it might be the ISBN #.
- The unique identifier is called the Primary Key. Every table must have one.
- Very often, you can tell the database system to automatically come up with its own unique number. That is called a Surrogate Key.
- Every other field in a table must be dependent on or describe the instance identified by the PK.
- SQL will let you create a table without a Primary Key, but without one the database can't tell rows apart

Foreign Key

- If a table connects to another table, it will do so using a Foreign Key.
- Terrible name, but it means the field connects to a Primary Key in another table.

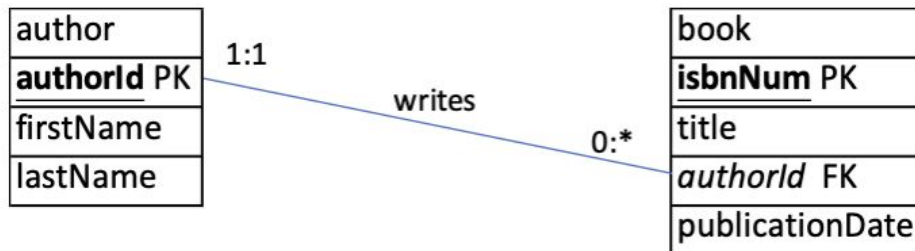
Relationship with Keys

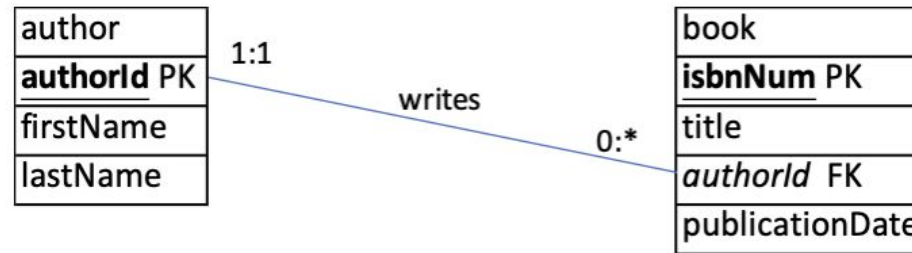


- authorId is the Primary Key of the author table
- isbnNum is the Primary Key of the book table
- authorId is a Foreign Key in the book table
- Primary Key is in bold and underlined
- Foreign Key is in italics

Cardinality

- Every relationship has cardinality. That means the range of things on each side.
- The numbers can be none, one or many
- Relationships can be one to many, one to one or many to many

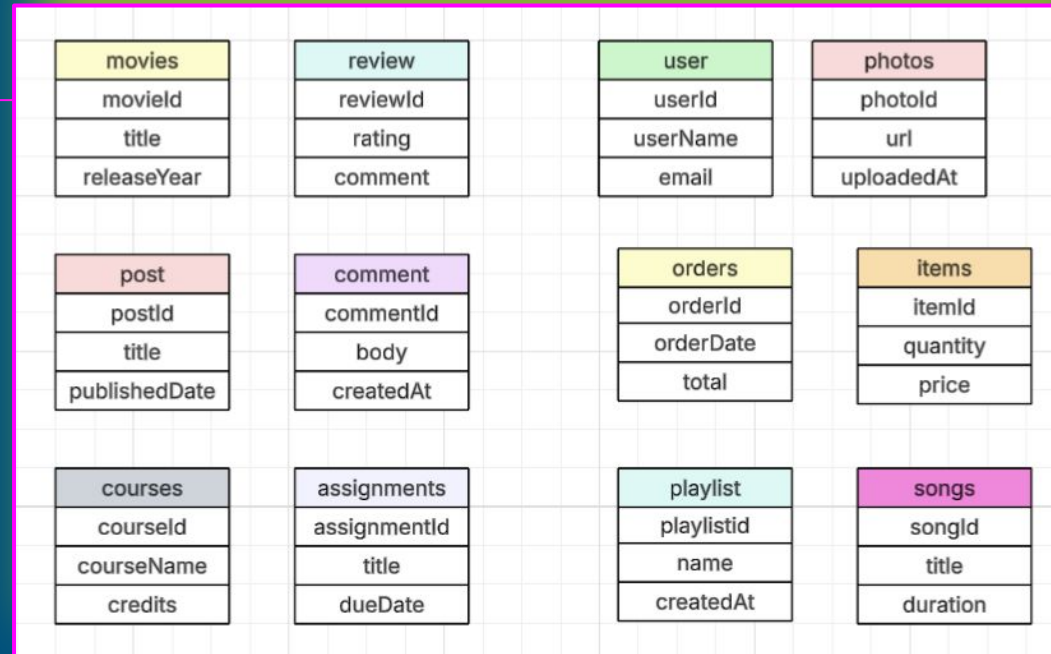




this is a classic
one-to-many
relationship example

- Cardinality is read from each side - “For one row in the other table, how many rows of this table can be related?”
- Author writes Book - to determine the cardinality we ask:
 - For one author, how many books can they write?
 - For one book, how many authors can it have?
- 1:1 (minimum 1: maximum 1) For one book, there must be exactly one author (this model assumes this, we can tell because there is only one column for authorId in the schema)
- 0:* (minimum 0 : maximum many) For one author, there can be zero, one, or many books (not written any in the db, has one, or has many)

Exercises in Lucid App



- What is the primary key of each table?
- What relationship likely exists between these two tables?
- What is the cardinality of this relationship?
- Where should the foreign key go?
- What would the foreign key column be named?

Example: GitHub Repository Database Schema



GitHub has a number of users who own and contribute to repositories. Each repository has a name, a unique identifier, and is owned by exactly one user. A user may own zero, one, or many repositories. Each repository contains a number of commits. Each commit has a unique identifier, a message, and a timestamp, and is associated with exactly one repository and exactly one author. A user may author many commits or may not have authored any commits yet. Users may also collaborate on repositories they do not own, and a repository may have multiple collaborators.

Bridging Tables

- You cannot implement many to many relationships in a Relational Database with a single foreign key
- Therefore, they are converted to two one to many relationships using a bridging table

